

Information Retrieval System

dengan Word2Vec Query Expansion

Kelompok 7

Orvin Andika Ikhsan Abhista (13523017)

Fajar Kurniawan (13523027)

Salman Hanif (13523056)

Aryo Wisanggeni (13523100)

Sabilul Huda (13523072)

June 3, 2026



[PLACEHOLDER_1: Foto Anggota Kelompok]

Figure 1: Foto Anggota Kelompok

Contents

1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Tujuan	3
1.3	Ruang Lingkup	3
2	Deskripsi Aplikasi	4
2.1	Gambaran Umum	4
2.2	Fitur Utama	4
2.3	Arsitektur Sistem	5
3	Deskripsi Program	7
3.1	Struktur Proyek	7
3.2	Modul Program	7
3.2.1	preprocessor.py	7
3.2.2	parser.py	8
3.2.3	indexer.py	8
3.2.4	word2vec_engine.py	8
3.2.5	query_expansion.py	8
3.2.6	evaluator.py	9
3.3	Alur Program	9
4	Library dan Dependencies	10
4.1	Python Packages	10
4.2	Word2Vec Model	10
4.3	Frontend	10
5	Hasil Eksperimen	11
5.1	Setup Eksperimen	11
5.2	Metrik Evaluasi	11
5.2.1	Average Precision (AP)	11
5.2.2	Mean Average Precision (MAP)	11
5.3	Hasil Overall	12
5.4	Hasil Per-Query	12
5.5	Analisis Perbandingan MAP Original vs Expanded	12
5.6	Screenshot Aplikasi	13
6	Pembagian Tugas	15
6.1	Anggota Kelompok	15
6.2	Detail Tugas	15

6.2.1	Programmer 1 - Orvin Andika Ikhsan Abhista (13523017)	15
6.2.2	Programmer 2 - Fajar Kurniawan (13523027)	15
6.2.3	Programmer 3 - Salman Hanif (13523056)	16
6.2.4	Programmer 4 - Aryo Wisanggeni (13523100)	16
6.2.5	Programmer 5 - Sabilul Huda (13523072)	16
7	Kesimpulan	17
7.1	Kesimpulan	17
7.2	Saran Pengembangan	17
8	Lampiran	18
8.1	Cara Menjalankan Program	18
8.1.1	Instalasi	18
8.1.2	Menjalankan API Server	18
8.1.3	Menjalankan Evaluasi	18
8.1.4	Menjalankan dengan Query Expansion	18
8.2	API Endpoints	18
8.3	Referensi	19
8.4	Foto Anggota	19

Chapter 1

Pendahuluan

1.1 Latar Belakang

Information Retrieval (IR) adalah proses pengambilan dokumen yang relevan dari koleksi besar berdasarkan query pengguna. Sistem IR konvensional menggunakan pencocokan kata kunci yang sering kali gagal menangkap makna semantik dari query.

Word2Vec adalah model embedding kata yang merepresentasikan kata dalam ruang vektor berdimensi tinggi, di mana kata-kata dengan makna serupa berada berdekatan. Dengan memanfaatkan Word2Vec untuk query expansion, sistem dapat menambahkan kata-kata semantik yang relevan ke query awal, sehingga meningkatkan recall tanpa mengorbankan precision secara signifikan.

Proyek ini mengimplementasikan sistem IR dengan fitur query expansion berbasis Word2Vec menggunakan dataset CISI (Collection of Scientific Information) yang berisi 1460 dokumen dan 112 query uji.

1.2 Tujuan

1. Mengimplementasikan sistem IR berbasis TF-IDF cosine similarity
2. Mengintegrasikan Word2Vec untuk semantic query expansion
3. Mengevaluasi performa sistem menggunakan Mean Average Precision (MAP)
4. Membandingkan hasil retrieval antara query original dan query expanded

1.3 Ruang Lingkup

- Dataset: CISI collection (1460 dokumen, 112 query, 76 query dengan qrels)
- Preprocessing: Lowercasing, tokenization, stopwords removal, Porter stemming
- Ranking: TF-IDF dengan cosine similarity
- Query Expansion: Word2Vec Google News 300 dimensions
- Evaluasi: Mean Average Precision (MAP)

Chapter 2

Deskripsi Aplikasi

2.1 Gambaran Umum

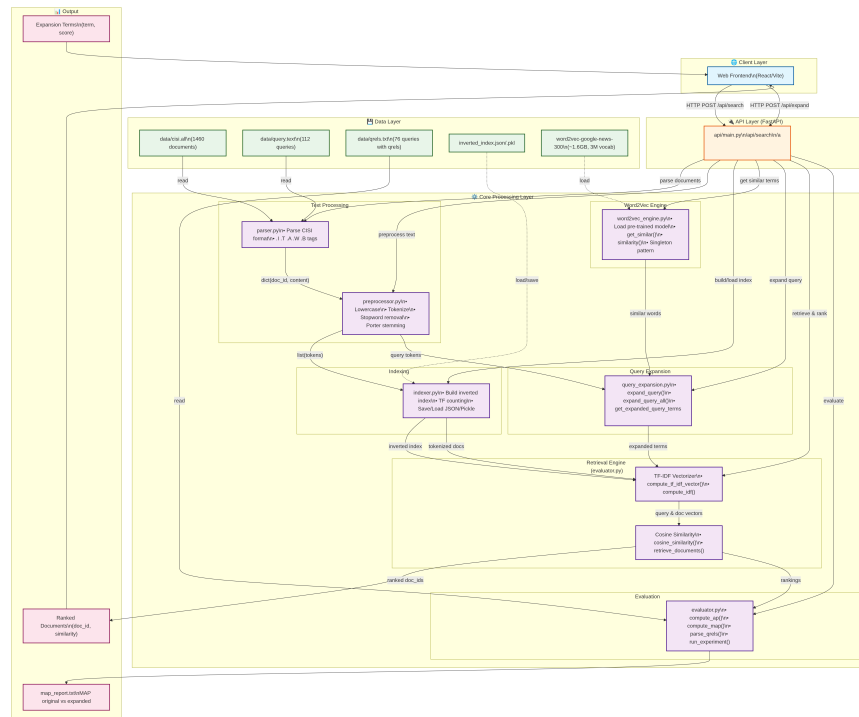
Sistem ini adalah aplikasi Information Retrieval yang memungkinkan pengguna untuk:

- Mencari dokumen dari koleksi CISI menggunakan query teks
- Melihat hasil retrieval dengan dan tanpa query expansion
- Mengeksplorasi term-term semantik yang ditambahkan oleh Word2Vec
- Melihat evaluasi MAP untuk perbandingan performa

2.2 Fitur Utama

1. **Document Parsing** - Parser untuk format dokumen CISI (.I, .T, .A, .W, .B tags)
2. **Text Preprocessing** - Normalisasi, tokenisasi, stopwords removal, stemming
3. **Inverted Index** - Pembangunan indeks terbalik untuk pencarian efisien
4. **TF-IDF Retrieval** - Ranking dokumen menggunakan TF-IDF cosine similarity
5. **Word2Vec Query Expansion** - Penambahan term semantik menggunakan model Word2Vec
6. **MAP Evaluation** - Evaluasi performa menggunakan Mean Average Precision
7. **REST API** - Endpoint API menggunakan FastAPI
8. **Web Interface** - Frontend interaktif (React/Vite)

2.3 Arsitektur Sistem



[PLACEHOLDER_2: Diagram Arsitektur Sistem]

Figure 2.1: Diagram Arsitektur Sistem

Alur data sistem:

1. User memasukkan query melalui web interface
2. Query dipreprocess (tokenize, stem, remove stopwords)
3. Word2Vec engine mencari term semantik serupa
4. Query expanded digunakan untuk retrieval TF-IDF
5. Hasil diurutkan berdasarkan cosine similarity

6. Hasil ditampilkan dengan perbandingan original vs expanded

Chapter 3

Deskripsi Program

3.1 Struktur Proyek

```
1 InformationRetrievalSystem-Word2Vec/  
2 +-- src/  
3 |   +-- __init__.py  
4 |   +-- preprocessor.py           # Text preprocessing  
5 |   +-- parser.py                 # CISI document parser  
6 |   +-- indexer.py                # Inverted index builder  
7 |   +-- word2vec_engine.py        # Word2Vec model wrapper  
8 |   +-- query_expansion.py        # Query expansion logic  
9 |   \-- evaluator.py              # MAP evaluation  
10 +-- api/  
11 |   \-- main.py                  # FastAPI REST API  
12 +-- frontend/                   # React web interface  
13 +-- data/  
14 |   +-- cisi.all                  # Document collection  
15 |   +-- query.text                # Test queries  
16 |   \-- qrels.txt                 # Relevance judgments  
17 +-- main.py                      # Entry point  
18 \-- pyproject.toml               # Dependencies
```

Listing 3.1: Struktur Direktori

3.2 Modul Program

3.2.1 preprocessor.py

Modul ini menangani preprocessing teks dengan fungsi-fungsi:

- `preprocess(text, stemming, remove_stopword)` - Fungsi utama preprocessing
- `_normalize_text(text)` - Lowercase dan hapus karakter non-alfanumerik
- `_tokenize(text)` - Split teks menjadi token
- `_remove_stopwords(tokens)` - Hapus stopwords English (NLTK)

- `_stem_tokens(tokens)` - Porter Stemmer

3.2.2 `parser.py`

Parser untuk format dokumen CISI:

- `parse_docs(file_path)` - Parse dokumen dari file
- `parse_text(raw)` - Parse dokumen dari string
- Tags yang didukung: `.I` (ID), `.T` (Title), `.A` (Author), `.W` (Abstract), `.B` (Bibliography)
- Output: `dict[doc_id, content]`

3.2.3 `indexer.py`

Pembangunan inverted index:

- `build_index(docs)` - Build inverted index dari koleksi dokumen
- `save_index(index, file_path)` - Simpan index ke JSON/Pickle
- `load_index(file_path)` - Load index dari file
- `get_document_terms(index, doc_id)` - Get terms untuk dokumen tertentu

3.2.4 `word2vec_engine.py`

Wrapper untuk model Word2Vec:

- `init_engine()` - Inisialisasi singleton engine
- `get_engine()` - Get engine instance
- `Word2VecEngine.get_similar(term, top_n)` - Get semantically similar terms
- `Word2VecEngine.similarity(term1, term2)` - Cosine similarity dua kata
- `Word2VecEngine.train()` - Train model dari koleksi dokumen
- Filter: lowercase only, no underscores, min length 3, no substring matches

3.2.5 `query_expansion.py`

Logika query expansion:

- `expand_query(query_terms, top_n)` - Expand dengan batas term
- `expand_query_all(query_terms)` - Expand tanpa batas
- `get_expanded_query_terms()` - Get combined original + expanded terms
- `format_expansion_detail()` - Format output untuk logging

3.2.6 evaluator.py

Evaluasi performa sistem:

- `parse_qrels(file_path)` - Parse relevance judgments (auto-detect format)
- `compute_ap(rankings, relevant_docs)` - Average Precision per query
- `compute_map(rankings, qrels)` - Mean Average Precision
- `run_experiment()` - Run full experiment (original vs expanded)
- `write_report(results)` - Write evaluation report

3.3 Alur Program

1. **Inisialisasi:** Load Word2Vec model via `init_engine()`
2. **Parsing:** Parse dokumen CISI menjadi `dict[doc_id, content]`
3. **Preprocessing:** Tokenize, stem, remove stopwords
4. **Indexing:** Build inverted index dengan term frequencies
5. **IDF Computation:** Hitung IDF untuk setiap term
6. **Query Processing:**
 - (a) Preprocess query
 - (b) Expand query dengan Word2Vec (opsional)
 - (c) Compute TF-IDF vector untuk query
7. **Retrieval:** Compute cosine similarity dengan semua dokumen
8. **Ranking:** Urutkan dokumen berdasarkan similarity score
9. **Evaluation:** Hitung MAP menggunakan qrels

Chapter 4

Library dan Dependencies

4.1 Python Packages

Package	Versi	Kegunaan
fastapi	$\geq 0.136.3$	REST API framework
uvicorn	$\geq 0.48.0$	ASGI server
gensim	$\geq 4.4.0$	Word2Vec model implementation
nltk	$\geq 3.9.4$	Tokenization, stopwords, Porter stemmer
pysastrawi	$\geq 1.2.1$	Indonesian stemmer (opsional)
pytest	$\geq 9.0.3$	Testing framework
python-multipart	$\geq 0.0.30$	Form data parsing

Table 4.1: Dependencies Python

4.2 Word2Vec Model

Model yang digunakan: `word2vec-google-news-300`

- Dimensi vektor: 300
- Training corpus: Google News (100 miliar kata)
- Vocabulary: 3 juta kata
- Ukuran file: 1.6 GB
- Format: KeyedVectors (binary)

4.3 Frontend

[PLACEHOLDER_3: Detail framework frontend (React/Vite, versi, dependencies)]

Chapter 5

Hasil Eksperimen

5.1 Setup Eksperimen

- **Dataset:** CISI Collection
- **Jumlah Dokumen:** 1460
- **Jumlah Query:** 112 (76 dengan qrels)
- **Preprocessing:** Stemming = True, Stopword Removal = True
- **Ranking:** TF-IDF Cosine Similarity
- **Retrieval:** Top-100 documents per query
- **Query Expansion:** Top-5 similar terms per query term

5.2 Metrik Evaluasi

5.2.1 Average Precision (AP)

Average Precision mengukur kualitas ranking untuk satu query:

$$AP = \frac{1}{|R|} \sum_{k=1}^n P(k) \times rel(k) \quad (5.1)$$

dimana:

- $|R|$ = jumlah dokumen relevan
- $P(k)$ = precision pada posisi ke- k
- $rel(k) = 1$ jika dokumen ke- k relevan, 0 jika tidak

5.2.2 Mean Average Precision (MAP)

Mean Average Precision adalah rata-rata AP dari semua query:

$$MAP = \frac{1}{|Q|} \sum_{q=1}^{|Q|} AP(q) \quad (5.2)$$

dimana $|Q|$ = jumlah query.

5.3 Hasil Overall

Metrik	Original	Expanded
MAP	0.1791	0.1791

Table 5.1: Hasil MAP Overall

Catatan: Hasil di atas dijalankan tanpa query expansion (Word2Vec model belum di-load). Untuk hasil dengan query expansion, jalankan eksperimen dengan `use_expansion=True` setelah menginisialisasi Word2Vec engine.

5.4 Hasil Per-Query

Query	AP Original	Relevant Docs
1	0.2809	46
2	0.0094	26
3	0.2216	44
4	0.0674	8
5	0.0162	24
10	0.2796	26
11	0.0727	127
20	0.0319	144
44	0.1110	155
50	0.3016	89
52	0.7053	15
55	0.8625	18
62	0.5986	12
65	0.4708	13
66	0.4796	35
97	0.5507	6
102	0.4578	24
109	0.2014	71
111	0.5788	6

Table 5.2: Sample Hasil AP Per-Query (Top Queries)

5.5 Analisis Perbandingan MAP Original vs Expanded

[PLACEHOLDER_4: Analisis perbandingan MAP original vs expanded query]

Template analisis yang perlu diisi setelah menjalankan eksperimen dengan Word2Vec:

1. Apakah query expansion meningkatkan MAP? Berapa persen perubahannya?
2. Query mana yang mengalami peningkatan signifikan? Mengapa?

3. Query mana yang mengalami penurunan? Mengapa?
4. Apakah ada pola tertentu pada query yang beneficia dari expansion?
5. Bagaimana trade-off antara recall dan precision?

5.6 Screenshot Aplikasi

[PLACEHOLDER_5: Screenshot Halaman Utama]

Figure 5.1: Tampilan Halaman Utama Aplikasi

[PLACEHOLDER_6: Screenshot Hasil Pencarian]

Figure 5.2: Tampilan Hasil Pencarian

[PLACEHOLDER_7: Screenshot Query Expansion]

Figure 5.3: Tampilan Query Expansion

Chapter 6

Pembagian Tugas

6.1 Anggota Kelompok

NIM	Nama	Role	File Utama
13523017	Orvin Andika Ikhsan Abhista	Programmer 1	preprocessor.py, parser.py
13523027	Fajar Kurniawan	Programmer 2	indexer.py
13523056	Salman Hanif	Programmer 3	word2vec_engine.py
13523100	Aryo Wisanggeni	Programmer 4	query_expansion.py
13523072	Sabilul Huda	Programmer 5	evaluator.py, Laporan

Table 6.1: Pembagian Tugas Anggota Kelompok

6.2 Detail Tugas

6.2.1 Programmer 1 - Orvin Andika Ikhsan Abhista (13523017)

Tanggung Jawab: Preprocessing dan Parsing

- Implementasi text normalization (lowercase, remove punctuation)
- Implementasi tokenization
- Integrasi NLTK stopwords removal
- Integrasi Porter Stemmer
- Parser format dokumen CISI (.I, .T, .A, .W, .B tags)
- Normalisasi document ID ke format DOC###

6.2.2 Programmer 2 - Fajar Kurniawan (13523027)

Tanggung Jawab: Inverted Index

- Build inverted index dari koleksi dokumen
- Term frequency counting

- Document frequency tracking
- Save/load index (JSON dan Pickle format)
- Document terms retrieval

6.2.3 Programmer 3 - Salman Hanif (13523056)

Tanggung Jawab: Word2Vec Engine

- Wrapper Word2Vec model (gensim)
- Load pre-trained model dari gensim downloader
- Train model dari koleksi dokumen
- Implementasi `get_similar()` dengan filtering
- Implementasi `similarity()` untuk dua kata
- Singleton pattern untuk engine instance

6.2.4 Programmer 4 - Aryo Wisanggeni (13523100)

Tanggung Jawab: Query Expansion

- Implementasi `expand_query()` dengan Word2Vec
- Implementasi `expand_query_all()` untuk unlimited expansion
- Implementasi `get_expanded_query_terms()` helper
- Format expansion detail untuk output
- Handling OOV (out-of-vocabulary) terms

6.2.5 Programmer 5 - Sabilul Huda (13523072)

Tanggung Jawab: Evaluasi dan Laporan

- Implementasi `compute_ap()` untuk Average Precision per query
- Implementasi `compute_map()` untuk MAP overall
- Parser qrels (auto-detect format binary/graded)
- Experiment runner (original vs expanded)
- Report generation
- Koordinasi penulisan laporan
- Programmer's Reference

Chapter 7

Kesimpulan

7.1 Kesimpulan

[PLACEHOLDER_8: Kesimpulan akhir setelah menjalankan eksperimen dengan query expansion]

Template yang perlu diisi:

- Apakah sistem IR berhasil diimplementasikan?
- Bagaimana performa sistem (MAP score)?
- Apakah query expansion memberikan peningkatan?
- Apa kelebihan dan kekurangan sistem ini?

7.2 Saran Pengembangan

1. Implementasi relevance feedback (Rocchio algorithm)
2. Penambahan ranking algorithm BM25
3. Support untuk dataset lain (Cranfield, Medline)
4. Optimasi performa dengan index caching
5. Docker containerization untuk deployment
6. Unit tests coverage $> 80\%$
7. UI/UX improvement pada frontend

Chapter 8

Lampiran

8.1 Cara Menjalankan Program

8.1.1 Instalasi

```
1 # Install dependencies
2 uv sync
3
4 # Download NLTK data
5 python -c "import nltk; nltk.download('stopwords'); nltk.
    download('punkt')"
```

8.1.2 Menjalankan API Server

```
1 uv run uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
```

8.1.3 Menjalankan Evaluasi

```
1 uv run python -m src.evaluator
```

8.1.4 Menjalankan dengan Query Expansion

```
1 from src.word2vec_engine import init_engine
2 from src.evaluator import run_experiment
3
4 init_engine() # Load Word2Vec model
5 results = run_experiment(use_expansion=True)
```

8.2 API Endpoints

- GET / - Health check
- POST /api/search - Search dengan query expansion

- POST `/api/expand` - Preview expansion terms
- GET `/api/index/{doc_id}` - Get inverted index
- GET `/api/map` - Get MAP evaluation report

8.3 Referensi

1. Mikolov, T., et al. (2013). "Efficient Estimation of Word Representations in Vector Space." ICLR.
2. Robertson, S.E., et al. (1994). "Okapi at TREC-3." TREC.
3. Manning, C.D., Raghavan, P., & Schütze, H. (2008). "Introduction to Information Retrieval." Cambridge University Press.
4. CISI Dataset: https://ir.dcs.gla.ac.uk/resources/test_collections/cisi
5. Gensim Documentation: <https://radimrehurek.com/gensim/>
6. NLTK Documentation: <https://www.nltk.org/>
7. FastAPI Documentation: <https://fastapi.tiangolo.com/>

8.4 Foto Anggota

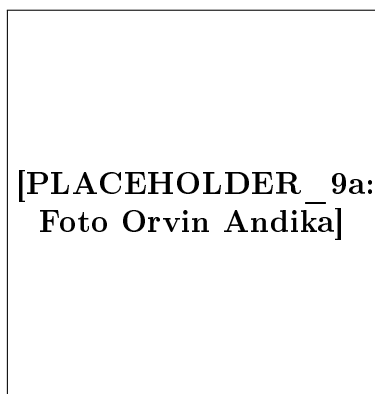


Figure 8.1: Orvin Andika Ikhsan Abhista (13523017)

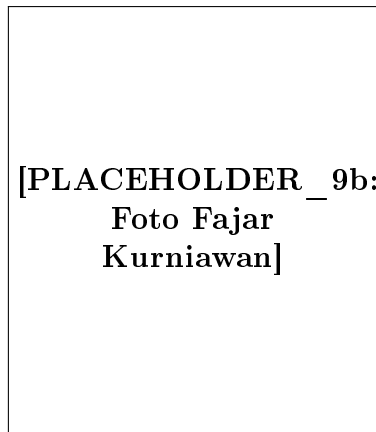


Figure 8.2: Fajar Kurniawan (13523027)

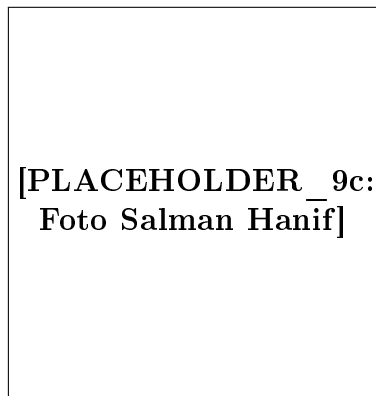


Figure 8.3: Salman Hanif (13523056)

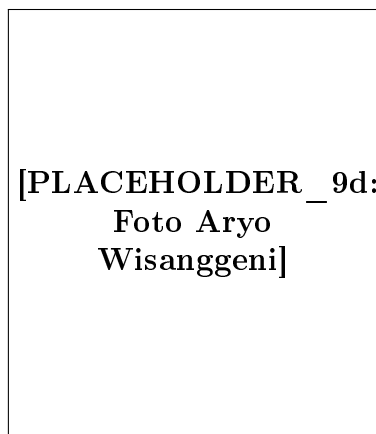


Figure 8.4: Aryo Wisanggeni (13523100)

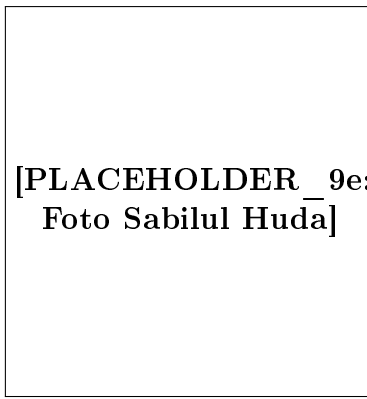


Figure 8.5: Sabilul Huda (13523072)